

Tarea 1

Bulk-loading de R-trees

Integrantes: Andrés Franz M.
Nahuel L. Sanhueza
Michelle Pérez G.

Auxiliares: Claudio Gaete
Gabriel Tapia

Profesores: Benjamín Bustos
Gonzalo Navarro

Fecha de entrega: 10 de Mayo de 2026
Santiago de Chile

Índice de Contenidos

1. Introducción	1
1.1. Resumen	1
1.2. Tema de Estudio	1
1.3. Hipótesis	1
2. Desarrollo	3
2.1. Convenciones Previas	3
2.2. Implementación de la búsqueda por rango	4
2.3. Implementación del Método Nearest-X	5
2.3.1. Funciones y estructuras	5
2.3.2. Funcionamiento de la implementación de Nearest-X	5
2.4. Implementación del Método Sort-Tile-Recursive (STR)	6
2.4.1. Funciones y estructuras	6
2.4.2. Funcionamiento de la implementación de Sort-Tile-Recursive	7
3. Resultados	9
3.1. Experimentos	9
3.2. Rendimiento Nearest-X	11
3.3. Rendimiento Sort-Tile-Recursive (STR)	12
3.4. Comparación entre métodos	13
4. Análisis	14
4.1. Tiempo de construcción	14
4.2. Operaciones de I/O simuladas	14
4.3. Consultas de rango	14
4.4. Comparación global	16
5. Conclusiones	17

Índice de Figuras

1. Tiempo de construcción del Árbol en función de N	11
2. Tiempo de construcción del Árbol en función de N	12
3. Tiempo de construcción del Árbol en función de N	13
4. Lecturas a disco	15
5. Puntos encontrados	15

Índice de Tablas

1. Comparativa global de rendimiento entre Nearest-X y STR. 16

1. Introducción

1.1. Resumen

En este informe se estudia el desempeño de dos métodos *bulk-loading* para R-trees: *Nearest-X* y *Sort-Tile-Recursive* (STR). Los R-trees son estructuras de datos espaciales utilizadas para almacenar objetos en múltiples dimensiones, en este caso puntos en dos dimensiones. Su funcionamiento se basa en agrupar los datos mediante rectángulos mínimos envolventes, o *Minimum Bounding Rectangles* (MBR), permitiendo responder consultas espaciales sin tener que revisar necesariamente todos los puntos del conjunto de datos.

El objetivo de este trabajo es implementar ambos métodos de construcción masiva y comparar su rendimiento en términos de tiempo de construcción y eficiencia de consultas por rango. Las consultas consideradas corresponden a rectángulos dentro del espacio normalizado $[0, 1] \times [0, 1]$ y deben retornar todos los puntos contenidos en dicho rectángulo.

1.2. Tema de Estudio

El tema de estudio se centra en la comparación de los métodos *Nearest-X* y *Sort-Tile-Recursive* para la construcción masiva de R-trees sobre puntos bidimensionales. Ambos métodos reciben el conjunto completo de puntos y construyen el árbol agrupando elementos en nodos de capacidad fija, pero se diferencian en la forma en que ordenan y distribuyen espacialmente dichos elementos.

Nearest-X ordena los MBRs según la coordenada X de su centro y luego forma grupos consecutivos. En cambio STR primero ordena por X , luego divide en franjas y finalmente ordena cada franja según la coordenada Y , buscando generar agrupaciones más compactas espacialmente.

La comparación se realizará considerando dos datasets: uno con puntos aleatorios uniformemente distribuidos y otro con puntos reales correspondientes a ubicaciones de edificios en Europa, ambos normalizados al rango $[0, 1] \times [0, 1]$. Las métricas principales serán el tiempo de construcción del árbol, la cantidad promedio de lecturas a disco por consulta, la cantidad promedio de puntos encontrados y la variabilidad en los resultados de las consultas.

1.3. Hipótesis

Se plantea como hipótesis que el método STR presentará un mejor rendimiento en las consultas por rango, especialmente en el número de lecturas desde disco. Esto se debe a que STR considera tanto la coordenada X como la coordenada Y al momento de agrupar los elementos, lo que debería producir MBRs más compactos y con menor traslape espacial, por lo que una consulta rectangular debería descender por menos ramas del árbol y por lo tanto

leer menos nodos desde disco.

Por otra parte se espera que *Nearest-X* pueda presentar una construcción más simple y posiblemente más rápida ya que su criterio principal consiste en ordenar únicamente por la coordenada X del centro de cada MBR. Sin embargo esto podría generar agrupaciones más alargadas o menos compactas afectando el rendimiento de las consultas.

Para evaluar esta hipótesis se implementaron ambos métodos en C++ y se diseñaron experimentos sobre los dos datasets indicados en el enunciado. Los resultados experimentales se presentan en las secciones posteriores del informe.

2. Desarrollo

2.1. Convenciones Previas

- El lenguaje de programación utilizado fue **C++**.
- El tamaño de página de disco se fijó en 4096 bytes. Cada nodo del R-tree corresponde exactamente a una página o bloque de disco.
- Se estableció $B = 204$ como la cantidad máxima de entradas por nodo. Esta capacidad permite que cada nodo tenga tamaño exacto de 4096 bytes.
- Las principales estructuras del R-tree se implementaron en **node.hpp**. Los puntos del dataset se representan mediante la estructura **Punto**, compuesta por dos valores **float**:

```
struct Punto {  
    float x;  
    float y;  
};
```

- Los rectángulos mínimos envolventes se representan mediante la estructura **MBR**:

```
struct MBR {  
    float x1;  
    float x2;  
    float y1;  
    float y2;  
};
```

En esta representación, **x1** y **x2** corresponden al límite inferior y superior en la coordenada *X*, mientras que **y1** y **y2** corresponden al límite inferior y superior en la coordenada *Y*.

- Cada entrada del nodo se representa mediante la estructura **Entrada**. Esta estructura contiene el MBR asociado y el índice del nodo hijo:

```
struct Entrada {  
    MBR mbr;  
    int32_t hijo;  
};
```

Si la entrada pertenece a una hoja el campo **hijo** toma el valor -1, indicando que no existe un nodo hijo asociado.

- Cada nodo del R-tree se representa mediante la estructura **Nodo**:

```
struct Nodo {  
    int32_t k;  
    Entrada entradas[B];  
    char pad[12];  
};
```

El campo **k** indica cuántas entradas válidas contiene el nodo. El arreglo **entradas** almacena hasta $B = 204$ pares MBR-hijo. El campo **pad** se utiliza como relleno para completar exactamente los 4096 bytes requeridos por nodo.

- Para las operaciones sobre archivos binarios se definieron funciones en **disk.hpp** y **disk.cpp**. Estas funciones permiten leer puntos desde los datasets, escribir el vector de nodos del árbol en un archivo binario y leer nodos específicos desde dicho archivo.
- Para contar operaciones de entrada/salida se definió la estructura **IOStats**:

```
struct IOStats {  
    uint64_t lecturas;  
    uint64_t escrituras;  
};
```

Este contador se utiliza para registrar la cantidad de nodos leídos durante las consultas sobre el archivo serializado.

- Las funciones auxiliares comunes del R-tree, como el cálculo de MBRs, la intersección entre rectángulos y la búsqueda por rango se implementaron en **rtree.hpp** y **rtree.cpp**.

2.2. Implementación de la búsqueda por rango

La búsqueda por rango fue implementada en la función **buscar_rango**, ubicada en **rtree.cpp**. Esta función recibe la ruta del archivo binario (que contiene el árbol serializado), un MBR de consulta y un puntero opcional a **IOStats** para retornar la cantidad de lecturas realizadas.

La búsqueda comienza desde el nodo raíz en la posición 0 del archivo. Si el nodo leído es una hoja se revisan todas sus entradas y se agregan al resultado aquellas cuyo punto esté contenido dentro del rectángulo de consulta. Como los puntos se almacenan como MBRs degenerados, solo se necesita verificar si las coordenadas **x1** e **y1** se encuentran dentro de los límites de la consulta.

Si el nodo leído es interno se revisa cada una de sus entradas, para cada entrada se compara el MBR almacenado con el rectángulo de consulta. Si ambos rectángulos intersectan entonces existe la posibilidad de que el subárbol correspondiente contenga puntos dentro de la consulta, por lo que se lee recursivamente el nodo hijo asociado. En caso contrario ese subárbol se descarta completamente.

2.3. Implementación del Método Nearest-X

El método Nearest-X fue implementado en la función `construir_nearest_x`. Esta función recibe un vector de puntos y retorna un vector de nodos que representa el R-tree construido en memoria. La construcción se realiza respetando la convención del proyecto de dejar siempre la raíz en la posición 0 del vector, mientras que los demás nodos se almacenan de forma secuencial desde la posición 1 en adelante.

2.3.1. Funciones y estructuras

- `std::vector<Nodo>`: estructura utilizada para almacenar el árbol completo en memoria. Cada posición del vector representa un nodo del R-tree.
- `std::vector<Entrada>`: estructura auxiliar utilizada para representar las entradas del nivel que se está procesando. Estas entradas pueden corresponder inicialmente a puntos o, en niveles superiores, a MBRs que apuntan a nodos hijos.
- `crear_nodo()`: función auxiliar que crea un nodo vacío, inicializando su cantidad de entradas en cero.
- `crear_entrada_punto(punto)`: función auxiliar que transforma un punto del dataset en una entrada de hoja. En este caso, el MBR corresponde a un rectángulo degenerado, donde $x1 = x2$ e $y1 = y2$, y el índice del hijo es -1.
- `agregar_entrada(nodo, entrada)`: función auxiliar utilizada para insertar una entrada dentro de un nodo y actualizar su cantidad de elementos.
- `centro_x(mbr)`: función auxiliar utilizada para obtener la coordenada X del centro de un MBR. Esta función se usa como criterio principal de ordenamiento en Nearest-X.
- `calcular_mbr_nodo(nodo)`: función auxiliar que calcula el MBR mínimo que contiene todas las entradas almacenadas en un nodo.
- `crear_entrada_interna(mbr, indice_hijo)`: función auxiliar que crea una entrada interna a partir del MBR de un nodo y del índice donde dicho nodo fue guardado dentro del vector del árbol.

2.3.2. Funcionamiento de la implementación de Nearest-X

La implementación comienza creando un vector vacío de nodos llamado `arbol`. En primer lugar, se inserta un nodo vacío en la posición 0, la cual se reserva temporalmente para la raíz. Esto permite que los nodos hoja e internos que se creen durante el proceso puedan almacenarse desde la posición 1 en adelante, manteniendo la convención de que la raíz del R-tree siempre estará al inicio del archivo cuando el árbol sea serializado.

Luego, cada punto recibido como entrada se transforma en una entrada de hoja mediante `crear_entrada_punto`. Todas estas entradas se guardan en un vector llamado `nivel_actual`,

que representa el conjunto de elementos que deben agruparse en el nivel que se está construyendo.

Mientras `nivel_actual` tenga más de `B` entradas, se realiza una iteración del método Nearest-X. Primero, las entradas se ordenan según la coordenada `X` del centro de su MBR. Luego, se recorren en ese orden y se agrupan en bloques consecutivos de hasta `B = 204` entradas. Cada grupo se guarda dentro de un nuevo nodo, el cual se agrega al vector `arbol`. Antes de insertar el nodo, se registra la posición en la que quedará guardado, ya que ese índice será usado posteriormente como puntero desde el nodo padre.

Después de crear cada nodo, se calcula su MBR total usando `calcular_mbr_nodo`. Con ese MBR y el índice del nodo dentro del vector, se crea una entrada interna mediante `crear_entrada_interna`. Estas entradas internas se almacenan en un nuevo vector llamado `nivel_siguiente`, que representa el nivel inmediatamente superior del árbol.

Al terminar una iteración, `nivel_siguiente` pasa a ser el nuevo `nivel_actual`. De esta forma, el procedimiento se repite iterativamente: ordenar por `X`, agrupar en nodos, calcular MBRs y generar entradas para el nivel superior. El proceso termina cuando el nivel actual tiene a lo más `B` entradas, ya que todas ellas caben en un único nodo.

Finalmente, se crea el nodo raíz con las entradas restantes de `nivel_actual` y se guarda en `arbol[0]`, reemplazando el nodo vacío reservado al inicio. Con esto, el vector de nodos queda listo para ser escrito secuencialmente en disco y usado posteriormente en las consultas por rango.

2.4. Implementación del Método Sort-Tile-Recursive (STR)

El método Sort-Tile-Recursive fue implementado en la función `construir_str`. Al igual que Nearest-X, esta función recibe un vector de puntos y retorna un vector de nodos que representa el R-tree construido en memoria, manteniendo la raíz en la posición 0 del vector. La diferencia está en la forma en que se ordenan y agrupan las entradas antes de formar cada nivel del árbol.

STR primero ordena las entradas por la coordenada `X` del centro de su MBR y las divide en franjas. Luego, dentro de cada franja ordena las entradas por la coordenada `Y` del centro de su MBR y forma los grupos que se convertirán en nodos del R-tree.

2.4.1. Funciones y estructuras

- `std::vector<Nodo>`

Estructura utilizada para almacenar el árbol completo en memoria. Cada posición del vector representa un nodo del R-tree.

- **std::vector<Entrada>**
Estructura auxiliar utilizada para representar las entradas del nivel que se está procesando. Estas entradas pueden corresponder inicialmente a puntos o, en niveles superiores, a MBRs que apuntan a nodos hijos.
- **crear_nodo()**
Función auxiliar que crea un nodo vacío, inicializando su cantidad de entradas en cero.
- **crear_entrada_punto(punto)**
Función auxiliar que transforma un punto del dataset en una entrada de hoja. En este caso, el MBR corresponde a un rectángulo degenerado, donde $x_1 = x_2$ e $y_1 = y_2$, y el índice del hijo es -1.
- **agregar_entrada(nodo, entrada)**
Función auxiliar utilizada para insertar una entrada dentro de un nodo y actualizar su cantidad de elementos.
- **calcular_mbr_nodo(nodo)**
Función auxiliar que calcula el MBR mínimo que contiene todas las entradas almacenadas en un nodo.
- **crear_entrada_interna(mbr, indice_hijo)**
Función auxiliar que crea una entrada interna a partir del MBR de un nodo y del índice donde dicho nodo fue guardado dentro del vector del árbol.
- **comparar_por_centro_x(a, b)**
Función estática utilizada para ordenar las entradas según la coordenada X del centro de su MBR. En caso de empate, compara por la coordenada Y para garantizar un orden determinista.
- **comparar_por_centro_y(a, b)**
Función estática utilizada para ordenar las entradas según la coordenada Y del centro de su MBR. En caso de empate, compara por la coordenada X.

2.4.2. Funcionamiento de la implementación de Sort-Tile-Recursive

Funcionamiento de la implementación de Sort-Tile-Recursive

La implementación comienza creando un vector vacío de nodos llamado **arbol**. En primer lugar, se inserta un nodo vacío en la posición 0, la cual se reserva temporalmente para la raíz. Luego, cada punto recibido como entrada se transforma en una entrada de hoja mediante **crear_entrada_punto** y se almacena en un vector dinámico llamado **nivel_actual**, el cual representa el conjunto de elementos a agrupar en el nivel en construcción.

Mientras **nivel_actual** tenga más de B entradas (donde $B = 204$), se realiza una iteración del método STR. Primero, se calcula el número de franjas verticales necesarias mediante

la fórmula $S = \lceil \sqrt{n/B} \rceil$, donde n es la cantidad de elementos en el nivel actual. Con este valor, se determina el tamaño de cada franja vertical (**tamano_franja**), que agrupará $S \times B$ entradas.

A continuación, todos los elementos presentes en **nivel_actual** se ordenan globalmente según su coordenada X utilizando la función **comparar_por_centro_x**. Una vez que el nivel completo está ordenado horizontalmente, el algoritmo itera para procesar y subdividir cada una de las S franjas verticales.

Para cada franja, se calcula su índice de inicio y fin. Los elementos que pertenecen exclusivamente a dicha franja se ordenan localmente según su coordenada Y mediante la función **comparar_por_centro_y**. Luego, estos elementos (ya ordenados verticalmente) se recorren de manera secuencial y se agrupan en bloques consecutivos de hasta B entradas, formando así los nodos definitivos (tiles).

Cada vez que se llena un nodo, este se inserta en el vector **arbol** y se guarda el índice en el que quedó posicionado. Posteriormente, se calcula el MBR que envuelve a los elementos de ese nodo mediante **calcular_mbr_nodo** y se genera una entrada interna con **crear_entrada_interna**. Estas nuevas entradas se van almacenando en el vector **nivel_siguiete**, el cual contendrá los punteros del nivel inmediatamente superior del árbol.

Al terminar de procesar todas las franjas y construir los nodos correspondientes, el vector **nivel_siguiete** pasa a ser el nuevo **nivel_actual**. El ciclo se repite recursivamente construyendo el árbol de abajo hacia arriba, repitiendo el ordenamiento por X, la división en franjas y el agrupamiento por Y. El proceso concluye cuando el nivel actual se reduce a un tamaño menor o igual a B .

Finalmente, se crea el nodo raíz insertando en él las entradas restantes de **nivel_actual**. Este nodo se asigna directamente en **arbol[0]**, reemplazando el espacio vacío que se reservó al comienzo. Con esto, el vector de nodos queda estructurado correctamente bajo la heurística de STR y listo para ser serializado en disco.

3. Resultados

3.1. Experimentos

En esta sección se detalla la metodología experimental utilizada para evaluar el rendimiento de los métodos *bulk-loading* por Nearest-X y Sort-Tile-Recursive implementados. El objetivo fue comparar ambos métodos en dos etapas principales: construcción del R-tree y ejecución de consultas por rango sobre el árbol serializado en disco.

Los experimentos se realizaron sobre dos datasets de puntos bidimensionales normalizados al rango $[0, 1] \times [0, 1]$. El primero corresponde a puntos generados aleatoriamente con distribución uniforme mientras que el segundo corresponde a puntos reales asociados a ubicaciones de edificios en Europa. Para ambos datasets se utilizaron los primeros N puntos del archivo binario correspondiente.

Para la etapa de construcción se evaluaron los tamaños:

$$N \in \{2^{15}, 2^{16}, \dots, 2^{24}\}$$

Para cada valor de N se construyeron cuatro árboles: Random con Nearest-X, Random con STR, Europa con Nearest-X y Europa con STR. Cada árbol fue construido en memoria principal como un vector de nodos y luego serializado a un archivo binario en la carpeta **outputs/trees**. Esta etapa fue automatizada mediante el script **run_build_experiments.sh**, el cual ejecuta repetidamente el comando `./rtree build` para cada combinación de dataset, método y tamaño.

Las métricas registradas durante la etapa de construcción fueron:

- Dataset utilizado.
- Método de construcción: Nearest-X o STR.
- Cantidad de puntos utilizados: N .
- Cantidad de nodos generados.
- Tiempo de construcción en milisegundos.
- Cantidad de escrituras a disco realizadas al serializar el árbol.
- Ruta del archivo binario generado.

Los resultados de esta etapa fueron guardados en el archivo:

`outputs/results/build_times.csv`

Para la etapa de consultas se utilizaron los árboles construidos con $N = 2^{24}$. Sobre cada árbol se ejecutaron consultas rectangulares aleatorias dentro del espacio $[0, 1] \times [0, 1]$. Cada consulta corresponde a un cuadrado de lado s , considerando los siguientes tamaños:

$$s \in \{0.0025, 0.005, 0.01, 0.025, 0.05\}$$

Para cada combinación de dataset, método y tamaño de cuadrado se generaron 100 consultas aleatorias.

La ejecución de consultas fue automatizada mediante el script `run_query_experiments.py`. Este script llama al ejecutable en C++ mediante `./rtree query`, interpreta su salida y guarda los resultados en un archivo CSV.

Las métricas registradas para cada consulta fueron:

- Dataset utilizado.
- Método de construcción del árbol.
- Tamaño del dataset: N .
- Tamaño del lado del cuadrado consultado: s .
- Identificador de la consulta.
- Coordenadas del rectángulo consultado: $x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$.
- Cantidad de puntos encontrados.
- Cantidad de lecturas a disco realizadas durante la consulta.
- Tiempo de ejecución de la consulta en milisegundos.

Los resultados de esta etapa fueron guardados en el archivo:

`outputs/results/query_results.csv`

Los experimentos se implementaron íntegramente en C++. Los tiempos de ejecución se midieron con la librería estándar `<chrono>`.

Las pruebas fueron ejecutadas en un computador que cuenta con un procesador AMD Ryzen 5 2600 de 6 núcleos y 12 hilos a 3.4 GHz, y 16 GB de memoria RAM.

3.2. Rendimiento Nearest-X

Los resultados del experimento para la construcción y consultas se verán a continuación:

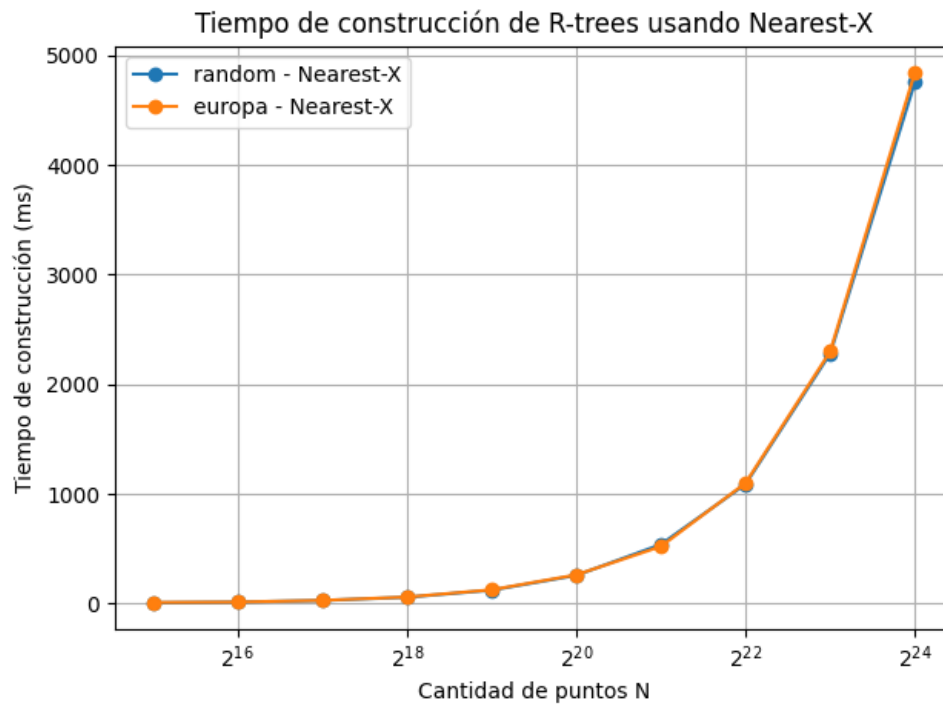


Figura 1: Tiempo de construcción del Árbol en función de N

3.3. Rendimiento Sort-Tile-Recursive (STR)

Los resultados del experimento para la construcción y consultas se verán a continuación:

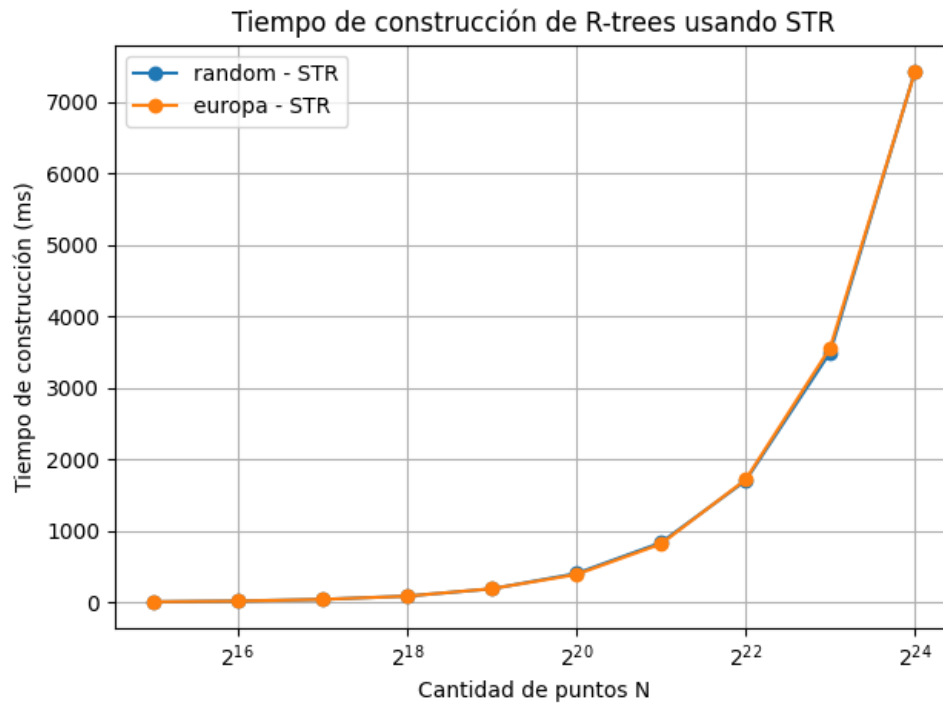


Figura 2: Tiempo de construcción del Árbol en función de N

3.4. Comparación entre métodos

Finalmente, se muestran gráficos comparativos entre ambos métodos:

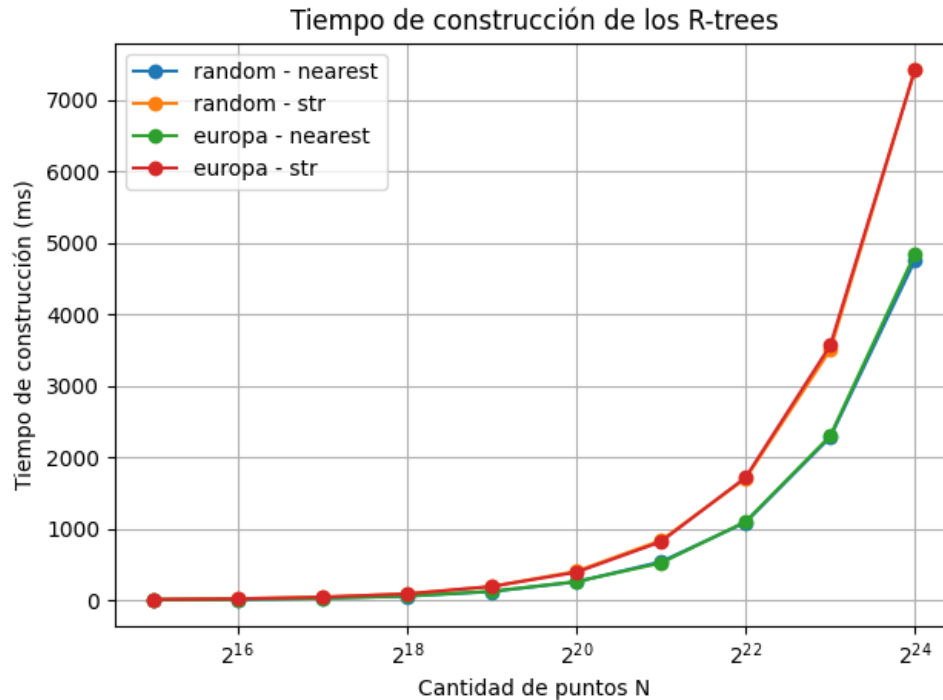


Figura 3: Tiempo de construcción del Árbol en función de N

El gráfico muestra los tiempos de construcción de los R-trees en función de N para las cuatro combinaciones evaluadas.

Se observa que STR es consistentemente más lento que Nearest-X, con un factor aproximado de 1.5x independiente del tamaño de entrada. Esta diferencia se explica por el trabajo adicional de ordenamiento que realiza STR: mientras Nearest-X solo ordena los datos por coordenada X, STR primero ordena globalmente por X y luego ordena cada franja por Y.

Ambos métodos escalan de forma similar, duplicando aproximadamente su tiempo al duplicar N , lo cual es consistente con la complejidad $O(n \log n)$ del ordenamiento.

No se observan diferencias significativas entre los datasets random y Europa, indicando que la distribución espacial de los puntos no afecta el costo de construcción. Las curvas de random y Europa para un mismo método prácticamente se superponen.

A pesar del mayor costo de construcción, STR produce árboles con mejor organización espacial, lo cual se traduce en consultas significativamente más eficientes.

4. Análisis

Los resultados obtenidos permiten evaluar el *trade-off* entre el costo de construcción y la eficiencia en consultas de los métodos Nearest-X y STR. Mientras STR requiere mayor tiempo de construcción, produce árboles con mejor organización espacial que se traduce en consultas significativamente más eficientes.

4.1. Tiempo de construcción

STR es aproximadamente 1.5x más lento que Nearest-X en la construcción, independiente del tamaño de entrada y del dataset utilizado. Para $N = 2^{24}$, Nearest-X construye el árbol en ~ 4.8 segundos mientras STR requiere ~ 7.4 segundos.

Esta diferencia se debe a que Nearest-X realiza un único ordenamiento por coordenada X, mientras que STR realiza un ordenamiento global por X seguido de múltiples ordenamientos por Y (uno por cada franja vertical). Ambos métodos escalan como $\mathcal{O}(n \log n)$.

La distribución espacial de los datos no afecta el tiempo de construcción: las curvas de random y Europa se superponen para cada método.

4.2. Operaciones de I/O simuladas

En construcción, ambos métodos generan la misma cantidad de nodos (82.649 para $N = 2^{24}$), ya que ambos agrupan de a $B = 204$ entradas por nodo.

La diferencia crítica aparece en las lecturas durante consultas. STR reduce drásticamente el número de accesos a disco:

- **Dataset random:** STR realiza entre 17x y 44x menos lecturas que Nearest-X.
- **Dataset Europa:** STR realiza entre 6x y 43x menos lecturas que Nearest-X.

Esta reducción se debe a que STR genera MBRs más compactos con menor solapamiento, permitiendo descartar más subárboles durante la búsqueda sin necesidad de visitarlos.

4.3. Consultas de rango

Se realizaron 100 consultas aleatorias para cada tamaño de cuadrado $s \in \{0.0025, 0.005, 0.01, 0.025, 0.05\}$.

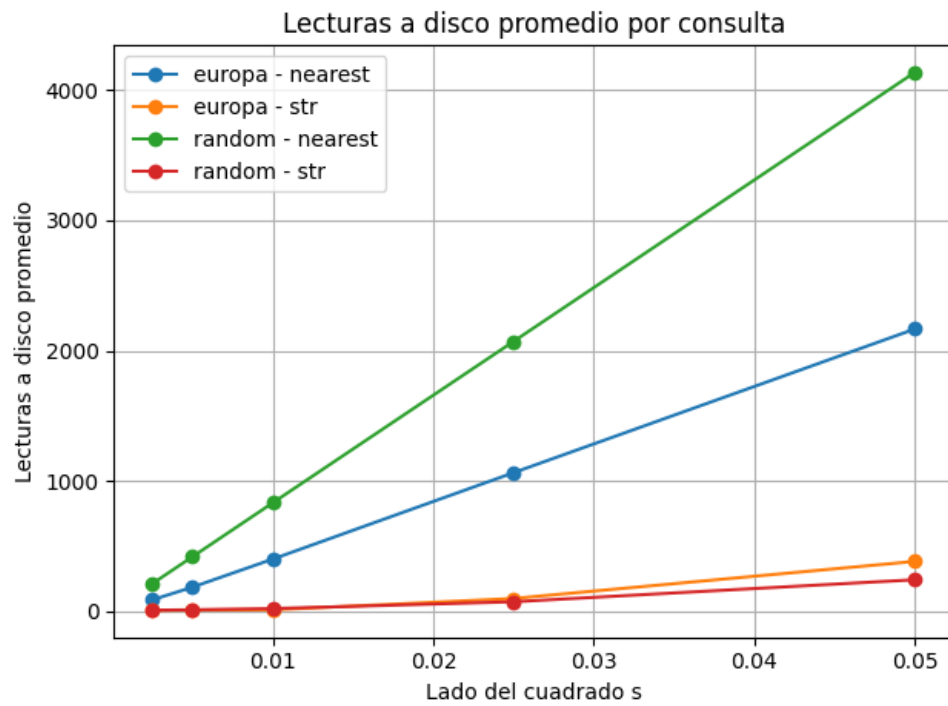


Figura 4: Lecturas a disco

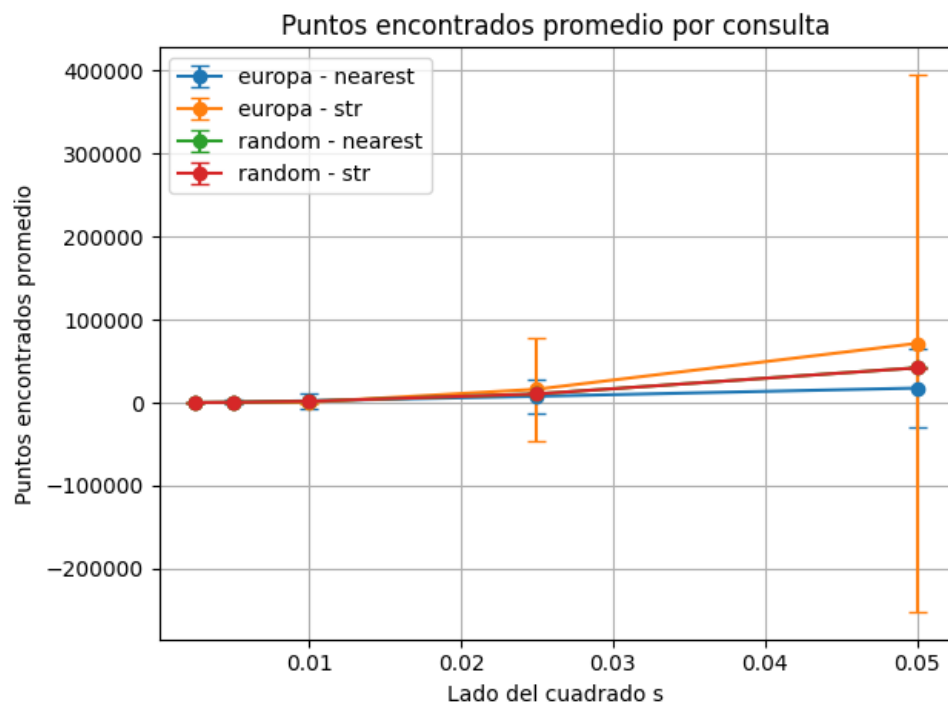


Figura 5: Puntos encontrados

- **Lecturas a disco:** El gráfico muestra que STR mantiene un número bajo de lecturas incluso para consultas grandes, mientras que Nearest-X crece rápidamente. Para $s = 0.05$, Nearest-X requiere ~ 4.100 lecturas en random vs ~ 240 de STR.
- **Puntos encontrados:** Ambos métodos retornan la misma cantidad de puntos (validando la correctitud). En el dataset random, la desviación estándar es baja debido a la distribución uniforme. En Europa, la desviación es alta porque la densidad de puntos varía según la región consultada (ciudades vs zonas rurales).

El tiempo de consulta está directamente correlacionado con las lecturas: STR responde entre 5x y 15x más rápido que Nearest-X.

4.4. Comparación global

Tabla 1: Comparativa global de rendimiento entre Nearest-X y STR.

Aspecto	Nearest-X	STR
Construcción	Más rápido (1x)	Más lento (1.5x)
Lecturas por consulta	Alto	Muy bajo (17-44x menos)
Tiempo de consulta	Más lento	Más rápido (5-15x)

El costo adicional de STR (~ 2.6 segundos extra para 16M puntos) se amortiza rápidamente: basta con realizar unas pocas decenas de consultas para que el ahorro en I/O compense el mayor tiempo de construcción. STR es la opción preferida cuando se realizan múltiples consultas sobre un mismo árbol.

5. Conclusiones

A partir de los resultados experimentales se puede concluir que la hipótesis planteada se cumple. El método STR presentó un mejor rendimiento en las consultas por rango, especialmente en la cantidad de lecturas a disco realizadas. Esto se debe a que STR agrupa los puntos considerando tanto la coordenada X como la coordenada Y , lo que genera MBRs más compactos y con menor solapamiento. Como consecuencia, durante una consulta se descartan más subárboles y se reduce la cantidad de nodos que deben ser leídos desde disco.

Por otro lado, Nearest- X tuvo un menor tiempo de construcción en comparación con STR. Esto era esperable, ya que su criterio de agrupamiento es más simple y se basa principalmente en ordenar los elementos según la coordenada X del centro de sus MBRs. Sin embargo, esta simplicidad produce agrupaciones menos compactas espacialmente, lo que afecta negativamente el desempeño de las consultas.

También se observó que ambos métodos generan la misma cantidad de nodos para un mismo valor de N , debido a que utilizan la misma capacidad máxima por nodo, $B = 204$. Por lo tanto, la diferencia entre ambos métodos no está en el tamaño final del árbol, sino en la forma en que los puntos son agrupados dentro de los nodos.

En términos generales, los resultados muestran un *trade-off* claro entre tiempo de construcción y eficiencia de consulta. Nearest- X puede ser conveniente si se busca construir rápidamente un árbol y realizar pocas consultas. En cambio, STR es preferible cuando el árbol será consultado muchas veces, ya que su mayor costo inicial de construcción se compensa con una reducción significativa en lecturas a disco y tiempo de búsqueda.

Como trabajo futuro, se podría extender la experimentación a otros tipos de consultas, por ejemplo rectángulos no cuadrados o consultas concentradas en regiones específicas del espacio. También sería interesante evaluar otros métodos de *bulk-loading*, analizar con mayor detalle el uso de memoria durante la construcción y repetir los experimentos varias veces para reducir el efecto de variaciones puntuales en los tiempos de ejecución. Finalmente, una posible extensión sería trabajar con el dataset de Europa no normalizado y visualizar consultas sobre ubicaciones geográficas reales.